

Cross-Site Scripting: A Dangerous and Preventable Software Error

Tyler Vander Mooren

INT 410: Secure Software

Dr. Linda Hamons

March 22, 2026

Cross-Site Scripting: A Dangerous and Preventable Software Error

What Is It?

Cross-site scripting also known as XSS, is a software weakness that occurs when a web application includes untrusted input in a page without properly neutralizing the input first. MITRE classifies this weakness as CWE-79, the improper neutralization of input during web page generation. In practice, XSS allows an attacker to inject code into a page that another user later loads. Because of how the browser treats that code as if it came from a trusted application, the attack may be able to steal session data, modify page content or even perform actions in the victim's session (MITRE, 2006). XSS is considered one of the top major software security problems by the SANS top 25 software errors. OWASP also explains that XSS can lead to account impersonation, loading malicious content, monitoring user activity and theft of sensitive information. This makes XSS more than just a minor front-end issue as it is a serious trust and execution problem inside the browser (OWASP, a ;SANS Institute).

Are There Similar Errors?

XSS appears in three commonly recognized forms: reflected, stored and DOM-based XSS. Reflected XSS happens when malicious input is immediately returned in a response page; an example would be a search result. Stored XSS happens when the payload is saved by the application and then later served to other users, such as through comments or a profile. DOM-based XSS is when a client-side JavaScript reads in untrusted data and then writes it into the page in an unsafe way. These are all a part of the same core weakness because it allows for untrusted data to become executable in the browser (MITRE, 2006; OWASP, b).

How Bad Is It?

The severity of an XSS vulnerability depends on the affected page and privileges of the victim. If an administrator were to visit a compromised page, then the attacker could potentially gain access to privileged functionality. Even when the victim is just a regular user the attacker may still steal session identifiers, inject phishing content or alter what the user sees and submits. Most commonly what occurs is a disclosure of private information that is stored in cookies including the session data (MITRE, 2006). XSS also affects all three parts of the CIA triad. Since if the data or session tokens are stolen, confidentiality is compromised. When the content of a user's session or transactions are modified it is breaking integrity. Availability gets affected when malicious scripts start to interfere with normal site use or damage user trust. For these reasons, XSS should be treated as a high-priority software weakness rather than a cosmetic bug (MITRE, 2006; OWASP, a).

How Do You Mitigate It?

The main defense against preventing XSS exploitation is to be aware of the output encoding and making sure it is properly neutralized. Any untrusted data must be encoded based on where it will appear, such as HTML, attributes, JavaScript, CSS or URLs (MITRE, 2006). There is no single encoding rule that can be generally applied that also works safely in every context. So, developers should avoid unsafe sinks such as innerHTML for when handling untrusted input. Input validation can help reduce exposure but it itself is not enough to stop XSS exploitation. Other measures such as Content Security Policy and secure cookie settings also help reduce impact but these measures do not replace safe output handling (OWASP, a, b). Below is an example of how XSS can occur and how it can be fixed. In the vulnerable code example below, user-controlled input from the query is directly inserted into the HTML response.

```
app.get("/search", (req, res) => {
  const term = req.query.term;
  res.send(`<h1>Results for: ${term}</h1>`);
});
```

This code is vulnerable because an attacker could supply a script through the input and the browser may execute it, for example using `<script>alert('XSS')</script>`. A safer version scrubs the input clean before including the value in the response and this can be seen in the code example below.

```
function escapeHtml(str) {
  return String(str)
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#39;");
}

app.get("/search", (req, res) => {
  const term = req.query.term;
  res.send(`<h1>Results for: ${escapeHtml(term)}</h1>`);
});
```

In this version, the browser cleans all input as text instead of potentially treating it as executable code. This works because the dangerous characters are encoded before entering the HTML context. In a real work environment, a framework with built-in auto-escaping and safe rendering patterns is usually preferable than relying entirely on a custom helper, however the main principle remains the same that untrusted input must be properly handled before it is placed into output (OWASP, a).

What policies, tests and/or processes would you put in place to prevent this when you are in charge?

Some immediate processes that would be implemented to prevent XSS would be instead of relying on individual developers to remember every rule to make it part of the normal

development process. This would be done by first creating a secure coding policy that forbids inserting user-controlled data directly into HTML, JavaScript, attributes or DOM sinks without approved encoding or framework protections. Also, unsafe patterns such as manually building HTML with request data or using innerHTML for untrusted content would be prohibited (OWASP, a, b). Additionally, peer review would be required for code that handles user input, template rendering, front-end DOM updates and authentication related features. Also, security testing would be integrated into the Secure Development Life Cycle (SDLC) following the National Institute of Standards and Technology's (NIST) framework recommendations. This framework emphasizes that secure development practices should be integrated into each SDLC implementation rather than as a one-time activity. Based on that approach, static testing would be required to identify unsafe output handling, dynamic testing with XSS payloads during QA and regression testing whenever an XSS flaw is found and fixed. Finally, further developer training to emphasize that input validation alone is not enough and that different outputs require different protections. This combination of policy, code review, testing and training is the most effective way to reduce repeated XSS issues over time (Souppaya et al., 2022).

Cross-site scripting remains one of the most dangerous and most preventable software errors in web development. It allows for untrusted input to become executable content inside a trusted browser, which can lead to session theft, account compromise, malicious content injection and data exposure. Although XSS is serious, it is also easily preventable when developers use context-aware encoding, safer frameworks, structured testing and clear secure development policies. The main lesson is that preventing XSS is not just about fixing a couple lines of code. It is about building a development process that will help eliminate these issues instead of introducing them.

References

- MITRE. (2006, July 19). *CWE-79: Improper Neutralization of Input During Web Page Generation* ("Cross-site Scripting"). CWE. <https://cwe.mitre.org/data/definitions/79.html>
- OWASP. (a). *Cross Site Scripting Prevention cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP. (b). *DOM based XSS Prevention cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html
- Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure software development framework (SSDF) version 1.1*. NIST Special Publication 800-218. National Institute of Standards of Technology (NIST). <https://doi.org/10.6028/nist.sp.800-218>
- SANS Institute. *Top 25 Software Errors*. <https://www.sans.org/top25-software-errors>